Round 2 Self-Audit — every pointer, graded honestly

The Why Layer · Accenture Innovation Challenge 2026 · BusinessIntelligence.ai

Audited 2 September 2026 against commit `a36439b`. 47 tests passing.

How to read this

Three verdicts, applied strictly:

VERDICT	MEANING
DELIVERED	Built, working, and covered by a test or a reproducible command.
PARTIAL	Something real exists, but a named part of the pointer is missing.
NOT BUILT	Absent. No code. Saying otherwise would be a lie.

I verified each claim against the source before grading. Where I found config that *looks* implemented but is never read by any code, I have said so — those are worse than gaps, because they read as features.

Headline: the 10 hard *Minimum Prototype Expectations* are 10/10 delivered. The gaps are concentrated in *Real-World Complexities* and *Solutioning Areas*, and there are **four dead-config items** that currently overstate the build.

Part A — The eight Round 2 objectives

1. Detects and prioritises material KPI movements

DELIVERED

Detection is `sift.py`. A median-based weekly seasonal index (so the anomaly cannot contaminate the baseline built to detect it), robust sigma from the MAD of pre-window residuals, then a **three-part materiality gate** from the contract: absolute rupee floor, share of period plan, and persistence in days. All four gates must pass.

Prioritisation is `triage.py --sweep`, which scans every KPI across every slice and ranks survivors by |z|, **and reports what it suppressed and under which gate**:

```
scanned 73 slices → 24 material, 9 data-quality, 16 insufficient history, 24 suppressed
suppressed because:
  persistence     20   did not persist for the required number of days
  statistical      8   inside the expected band once seasonality is removed
  pct_of_plan      4   movement is too small a share of the period plan
  abs_inr          1   below the rupee materiality floor in the contract
```

Honest caveat: "prioritises" here means ranking by statistical strength within a single window. There is no cross-KPI priority score that weighs, say, a ₹2 Cr revenue move against a 20pp service collapse. The worklist is sorted by |z|, which is not the same as sorted by business importance.

2. Reconciles data and business context across heterogeneous sources

PARTIAL — and this is the weakest of the eight.

What works. Four sources at three grains and four refresh cadences (60 / 1440 / 15 / 10080 minutes) are read through one governed access layer. Cross-source *referential* reconciliation is real and tested: shipments referencing orders that do not exist, shipments dated before their own order, CRM accounts absent from the

account master. Per-KPI `sliceable_by` declares which dimensions each source grain can actually resolve, so `otd_pct × segment` is reported as a grain limit rather than throwing.

What does not work — three dead-config items:

- `fiscal_calendar: "apr_mar"` is declared in the contract and **never read by any

code**. All windowing is plain Gregorian dates. An Indian FY client would get wrong period boundaries.

- `region: {hierarchy: [region, city]}` is declared and ** `city` does not exist in the

orders table at all**. There is no hierarchy traversal anywhere in the engine — no roll-up, no drill-down.

- There is **no reconciliation of conflicting KPI definitions**. The brief's real

complexity is two systems disagreeing on what "revenue" means. I have exactly one contract, so nothing is ever reconciled — the contract *asserts* a definition rather than resolving a conflict between two.

To branch from: add `city` to the generator and implement hierarchy roll-up; make the fiscal calendar actually drive period boundaries; introduce a second, conflicting source definition of net revenue and a documented resolution rule.

3. Identifies and ranks explanatory drivers using appropriate analytical methods

DELIVERED — the strongest part of the build.

Two independent mechanisms in `split.py`:

- **Identity algebra** — revenue = volume × price is exact, so the movement decomposes

into volume, mix and rate with **no model and no residual**. The identity closes to under ₹1, asserted by a test.

- **Dimension-lattice scan** — Adtributor-style (Bhagwan et al., NSDI 2014), ranked by

explanatory power weighted by share shift.

Then `evidence.py` grades each candidate on the Evidence Ladder — graph admissibility, temporal precedence against the declared lag, dose–response gradient (Spearman), corroboration with conflict scoring, and difference-in-differences with a placebo test. Ranking is share-weighted confidence, so a 10% mix effect that is L3 "for free" (it is an identity, not an inference) cannot outrank an 80% service failure at L2.

`propagation.py` adds the mechanism ledger — every hop measured against the same untreated cohort in its own lag-aligned window.

Honest caveat: "appropriate methods" in the brief includes **traditional ML**, and I use **none**. `MethodType.ML` exists in the telemetry enum and is **never emitted by any code path**. That is an empty category in my own method ledger.

4. Generates persona-specific narratives supported by traceable evidence

DELIVERED

Four personas, each with declared narrative depth, focus, word limit and channel. Same movement produces four genuinely different narratives — a test asserts three of them are byte-different. Every clause traces to the evidence packet; retrieved documents are cited with type, date, account and author role; lineage is on every response.

Honest caveat: narrative differentiation is currently driven by **template branching on `depth` and `focus`**, not by the model. With the deterministic narrator (the default, since no API key is configured) the four narratives differ in which sentences are assembled, not in register or voice. The LLM path would produce genuinely different prose, but the demo path does not.

5. Communicates uncertainty and abstains when evidence is insufficient or contradictory

DELIVERED — and this is what I would lead with.

Five distinct abstention states, each with a different reason and a different downstream behaviour: `COMPETING`, `UNKNOWN`, `INSUFFICIENT_HISTORY`, `DATA_QUALITY`, `ENTITLEMENT_DENIED`, plus `UNFIT_DATA` from the fitness gate.

Contradiction is measured, not assumed. Retrieved evidence supporting a *rival* theme is counted, and a conflict ratio above 0.4 marks the hypothesis **contested** and blocks L2:

SCENARIO	ON-THEME	CONFLICTING	RATIO	VERDICT
S1 dispatch SLA	12	0	0.00	corroborated → L3
S2 price rise	2	10	**0.83**	**contested — rung suppressed**
S6 competitor promo	11	0	0.00	corroborated → L3

When it abstains it prices the **cheapest separating test** — ₹1.8L, 14 days, owner CFO — so ambiguity becomes a next step rather than a hedge.

6. Recommends practical actions grounded in business levers, constraints and decision rights

DELIVERED

Recommendations are **retrieved from playbook memory** — past interventions with measured outcomes — matched on pattern signature and rescaled to the value now at risk. An action whose lever is not declared in the contract is **dropped before emission**, which is the mechanism that keeps recommendations inside real decision rights.

`delegation.py` then routes each recommendation to a human–AI collaboration mode derived from evidence grade × value at risk against the owner's authority limit × lever reversibility:

RECOMMENDATION	MODE	WHY
Counter-promotion (₹0.7L)	`AI_LED_HUMAN_APPROVES`	L3, reversible, inside authority
Service credit (₹44.6L)	`HUMAN_LED_AI_SUPPORTS`	L3, but **exceeds the RSM's ₹25L limit**
Price correction	`HUMAN_LED_AI_SUPPORTS`	hard to reverse
Under abstention	`HUMAN_ONLY_AI_ABSTAINS`	no cause established
—	`AI_DELEGATED`	**reserved, never assigned; a test enforces it**

Honest caveat: the playbook library has **three entries**. Pattern-matching against three seeds is a demonstration of the mechanism, not of institutional memory. Below a 0.45 similarity floor the engine emits nothing rather than dressing a generic suggestion as memory — which is correct behaviour but means coverage is thin.

7. Mechanism to learn from analyst and business-user feedback

PARTIAL

What works. Accept/reject writes to a store, updates a per-hypothesis prior weight (clamped to [0.35, 1.6]), and that weight multiplies into leader ranking on the next run — verified end-to-end by the audit harness. A recorded outcome re-estimates a playbook's effect size as a weighted mean, changing subsequent expected-impact figures.

Three real gaps:

- **The correction field is dead.** `feedback.record()` accepts a `correction` string and

the API exposes it, but **nothing ever reads it back**. There is no correction workflow — an analyst cannot say "the driver was actually X" and have the engine learn that.

- **There is no business-user feedback path.** The brief says "analyst *and business-user*

feedback". Only the analyst-facing accept/reject exists. A CFO cannot mark a narrative useless.

- **No expert validation workflow.** Nothing routes a disputed diagnosis to a reviewer.

To branch from: implement correction ingestion that amends the causal graph or adds a labelled counter-example; add a lightweight business-user signal (useful / not useful / wrong) distinct from the analyst grade.

8. Operates within realistic security, cost, latency and scalability constraints

PARTIAL — three of four.

- **Security — DELIVERED.** Row, column and domain entitlements enforced *before* prompt

assembly, with a packet-level leak test.

- **Cost — DELIVERED.** Per-run token and rupee ledger; ~₹0.30 per insight on Haiku 4.5.
- **Latency — DELIVERED.** 453 ms end-to-end, per-stage profile.
- **Scalability — NOT BUILT.** The estate is **45,065 order rows**. There is no load test,

no scale benchmark, no concurrency test, and no profiling above this size. DuckDB in-process on one file will not behave like a warehouse under concurrent users. Any scalability claim I made would be unfounded.

To branch from: generate a 10M-row estate and publish a latency curve; test the sweep under concurrency; document where the architecture would need to move server-side.

The LLM must not be the source of quantitative truth

DELIVERED — enforced in code, not asserted.

Two mechanisms. The model receives a **closed JSON evidence packet** with no tools and no data access, so it cannot compute. Afterwards a **numeric guard** parses every number in the output and matches it against that packet; unverifiable figures fail the narrative, which is retried once and then replaced by the deterministic narrator. Demonstrable live via `narrator=simulate_bad`.

Every step declares its method in a registry, producing the LLM/non-LLM split: `SQL 28 · RULES 10 · STATISTICS 4 · RETRIEVAL 3 · DETERMINISTIC 3 · CAUSAL 2` — **100% non-LLM offline, 91% with narration on.**

Honest caveats, two:

- The guard is **magnitude-tolerant at 1%** to survive rounding, so a fabricated figure

landing within 1% of a real one passes. In the demo, the invented ₹4.2 Cr is caught; an invented 9.7% that sits within tolerance of a real 9.64 is not. It is a strong filter on material fabrication, not a proof of correctness.

- The brief asks teams to demonstrate **traditional ML** among the method categories. I

demonstrate all of them **except ML**, which I use nowhere.

Part B — Real-World Complexities to Consider

• Multiple interacting drivers: price, volume, mix, marketing, supply, seasonality, competition, external events

PARTIAL — 7 of 8; marketing is missing

DRIVER	STATUS
Price	Identity rate component + `price_level` graph node + price-onset probe
Volume	Identity volume component + `order_volume` KPI
Mix	Identity mix component + `tier_mix` node; S1's second driver
Marketing	**NOT MODELLED.** No spend series, no campaign attribution. `market_events` carries an `own_promo` row that no hypothesis generator uses.
Supply	`warehouse_sla` → `delivery_delay`; the S1 primary driver
Seasonality	Robust weekly index in SIFT; differenced out by DiD
Competition	`competitor_promo` node; S2 and S6
External events	`market_events` source, weekly cadence

S1 demonstrates two *interacting* drivers (service failure 80% + mix shift 10%) separated cleanly. **To branch from:** add a marketing-spend series and a spend→volume edge.

• Different source-system refresh cadences, grains, data quality levels and historical coverage

DELIVERED — one of the strongest areas.

Four cadences (60 / 1440 / 15 / 10080 min), three grains (order line, shipment, event), per-source freshness SLAs with breach detection, deliberately varied quality (dispatch is both stale *and* partially loaded), and varied coverage (8 months for most, 19 days for the new category). `fitness.py` assesses five dimensions across all of it.

• Inconsistent KPI definitions, hierarchies, calendars, business rules and aggregation logic

PARTIAL — and honestly the weakest bullet in the whole brief for me

ELEMENT	STATUS
Aggregation logic	**DELIVERED.** `additive: true/false` per KPI; ratio metrics are never summed, always re-aggregated. This is a real trap avoided.
Business rules	**DELIVERED.** Materiality thresholds, lever authority limits, decision rights — all contract-driven.
Inconsistent definitions	**NOT BUILT.** One contract, no conflict, nothing reconciled.
Hierarchies	**NOT BUILT.** Declared as `[region, city]`; `city` is not in the data; no traversal exists.
Calendars	**NOT BUILT.** `fiscal_calendar: apr_mar` is declared and never read.

Three of five are dead or absent. This is the clearest place improvement should branch.

• Sparse history for new products, categories or markets

DELIVERED

S3: a category launched 2026-08-12 with 19 days of history. Below a 42-day floor the engine cannot fit a weekly seasonal index, so it falls back to a flat index, borrows a 22% coefficient-of-variation **peer-group prior**, inflates the interval 1.9×, and returns `INSUFFICIENT_HISTORY` with **no causal claim**. A near-zero sigma on a short series would otherwise produce absurd z-scores — that is guarded explicitly.

• Materiality based on both statistical significance and business impact

DELIVERED

Explicitly both, and they are separate gates. A movement must clear `statistical` (|z| ≥ warn sigma), **and** `abs_inr` (rupee floor), **and** `pct_of_plan` (share of period plan), **and** `persistence` (days). The sweep shows 8 slices suppressed on statistics and 5 on business impact in the same run — the two gates demonstrably do different work.

• Contradictory evidence, missing data and confidence calibration

PARTIAL — 2 of 3

- **Contradictory evidence — DELIVERED.** Conflict ratio, contested verdict, rung

suppression. See objective 5.

- **Missing data — DELIVERED.** Fitness gate catches stale feeds, partition-level load

failures and class-level load failures (the WH-4 case, where the *failure rows* stopped arriving while total volume held).

- **Confidence calibration — NOT BUILT.** Confidence is *composed* — ladder rank ×

playbook outcome confidence × pattern similarity — and **never validated**. Nobody has checked whether a stated 63% corresponds to a 63% success rate. There is no reliability diagram, no Brier score, no calibration curve. The number is a defensible composite, but calling it "calibrated" would be false.

To branch from: log predicted-vs-realised recovery per recommendation and publish a reliability curve once enough outcomes exist.

• Role-based personalization of insight depth, recommended actions and delivery channels

PARTIAL — 2 of 3

- **Depth — DELIVERED.** `executive` / `operational` / `technical` change what the

narrative contains.

- **Recommended actions — DELIVERED.** Different personas get different owners and

different delegation modes on the same movement.

- **Delivery channels — NOT BUILT.** Each persona declares a channel string — *"Monday

exec brief + email", "CRM task + WhatsApp digest"* — and **nothing delivers anything**. There is no email, no webhook, no scheduler, no queue. The channel is displayed as a label in the UI header. This currently reads as a feature and is not one.

• Row-, column- and domain-level security, sensitive-data protection and auditability

PARTIAL — security is strong, auditability is thinner than it looks

- **Row — DELIVERED.** Compiled into the WHERE clause; pre-flight refusal before any query

runs on an out-of-scope request.

- **Column — DELIVERED.** Denied measures redacted from the evidence packet itself; a test

asserts no rupee figure appears anywhere in the Supply Chain packet JSON.

- **Domain — DELIVERED.** CFO is denied CRM verbatims; withheld items are surfaced as a

count, never silently dropped.

- **Sensitive data — DELIVERED.** PII regex-redacted before retrieval results enter a

prompt.

- **Auditability — PARTIAL.** Full lineage, a method ledger with the actual SQL, and

per-stage telemetry are all present and genuinely strong. **But runs are not persisted.** There is no run archive, no immutable log, no way to reconstruct what the engine said last Tuesday. EU AI Act Art. 12 asks for post-hoc reconstruction of individual decisions; I can produce the *evidence* for a live run but cannot retrieve a past one.

To branch from: persist every run's evidence packet and telemetry to an append-only store keyed by `run_id`.

• Model and data drift, feedback capture and continuous evaluation

PARTIAL — 2 of 3, and drift is entirely absent

- **Feedback capture — DELIVERED** (with the correction gap noted in objective 7).
- **Continuous evaluation — PARTIAL.** `--backtest` replays 20 rolling historical windows

and scores precision 1.0 / recall 0.25 with zero false positives. That is real evaluation, but it is **run on demand, not continuously**, and it scores detection only — not diagnosis accuracy.

- **Drift — NOT BUILT.** No data drift detection (no PSI, no distribution comparison over

time), no model drift monitoring, no alert when the baseline stops resembling reality. The word "drift" appears once in the codebase, in an unrelated comment.

• LLM economics: model choice, token consumption, latency, caching, cost per insight

PARTIAL — 4 of 5; caching is the miss

- **Model choice — DELIVERED.** A routing policy in `llm_pricing.yaml` maps job type to

the cheapest model clearing the bar; per-model prices are configured.

- **Token consumption — DELIVERED.** Input/output tokens per call, per run.
- **Latency — DELIVERED.** Per-call ms and per-stage profile.
- **Cost per insight — DELIVERED.** USD and INR, ~₹0.30 per insight.
- **Caching — NOT BUILT.** The telemetry record has a `cached: bool` field that is

always False. No prompt cache, no result cache, no Anthropic prompt-caching headers. Repeat identical runs pay full price. The field's existence overstates the build.

Part C — Solutioning Areas ("could explore")

These are optional. Graded anyway, because pretending otherwise would defeat the purpose.

• Anomaly detection, contribution analysis, forecasting, causal inference, business-rule reasoning

PARTIAL — 4 of 5

Anomaly detection (robust control limits), contribution analysis (identity + lattice), causal inference (graph admissibility, precedence, dose–response, DiD with placebo) and business-rule reasoning (materiality, levers, authority) are all built and tested.

Forecasting — NOT BUILT. There is no forward projection anywhere. The "expected" line is a *trailing* robust level times a seasonal index — a backward-looking baseline, not a forecast. Nothing predicts what revenue will be next week, and there is no "what happens if we do nothing" trajectory.

• Governed KPI semantics, metadata, lineage, business rules, ontology or knowledge graphs

PARTIAL

Governed semantics, metadata, lineage and business rules are all delivered through the contract. The causal graph is **12 nodes and 12 edges with 4 explicit blocks** — genuinely useful, and honestly described it is a **small hand-curated DAG, not an ontology or a knowledge graph**. There is no entity resolution, no class hierarchy, no inference over relations beyond path search.

• LLM-assisted intent understanding, orchestration, narrative synthesis and contextual retrieval

PARTIAL — 1.5 of 4

- **Narrative synthesis — DELIVERED** (behind the guard).
- **Contextual retrieval — DELIVERED, but non-LLM.** BM25 scoped to segment + window +

mechanism lag. It is contextual, but the LLM plays no part in it.

- **Intent understanding — NOT BUILT.** There is no natural-language input anywhere. The

five scenarios are hard-coded dictionaries. A user cannot ask a question.

- **LLM orchestration — NOT BUILT, deliberately.** The pipeline is a fixed seven-stage

function. No agent, no planner, no tool-calling loop. This is a defensible choice — the control flow *is* the product's argument — but it is a choice, not an implementation.

• Proactive alerts, conversational analysis, augmented dashboards, decision workspaces

PARTIAL — 1 of 4

- **Proactive alerts — PARTIAL.** The sweep produces a ranked worklist, which is the

content of an alert. Nothing schedules it and nothing delivers it. No cron, no push.

- **Conversational analysis — NOT BUILT.** No chat interface, no follow-up questions, no

dialogue state.

- **Augmented dashboard — DELIVERED, weakly.** A single-page UI exists. It is a

demonstration surface, not a designed product, and I have said so in the developer guide.

- **Decision workspace — NOT BUILT.** No assignment, no status tracking, no collaboration,

no closing the loop on an action inside the tool.

• Confidence scoring, evidence citation, alternative hypotheses, abstention mechanisms

DELIVERED — all four, and the strongest cluster in the build.

Graded L0–L3 confidence with an explicit basis string; every retrieved document cited with type, date, account, author role; alternative hypotheses always shown side by side (a cognitive forcing function, per Buçinca et al. CSCW 2021); five distinct abstention states. **Caveat:** confidence is scored but not calibrated — see the earlier bullet.

• Action recommendations structured as: driver → controllable lever → action → expected impact → owner → confidence → monitoring plan

DELIVERED — exactly this structure, all seven fields.

```
driver:            Dispatch SLA failure at WH-3   (evidence L3)
lever:             service_credit                 (declared in contract)
action:            2% service credit on affected invoices, issued within 10 days
expected impact:   ₹44.6 L  — 71% of at-risk value, the rate this playbook achieved in 2024
owner:             regional_sales_manager         (authority limit ₹25 L)
confidence:        63%  — evidence L3 × playbook confidence 'high' × similarity 0.87
monitoring:        reorder_rate_28d, review in 42 days,
                   success = ≥43% of at-risk value recovered by week 6
```

Plus a delegation mode per recommendation, which the brief did not ask for.

• Human feedback, expert validation, correction workflows and learning loops

PARTIAL — 2 of 4

Human feedback and learning loops work (priors, measured outcomes). **Expert validation — NOT BUILT**: no review queue, no escalation, no second-opinion path. **Correction workflows — NOT BUILT**: the field is accepted and stored and never read.

• Platform-native / configured / custom-built / externally integrated distinction

PARTIAL

The build is **~95% custom** on DuckDB, with the Anthropic API as the one external integration. The README maps each component to its native equivalent (dbt Semantic Layer, Snowflake row-access policies, Cortex Search, Databricks Model Serving).

But the brief explicitly asks teams to *distinguish* native / configured / custom-built / externally integrated, and I have not produced that classification. Here it is, for the first time:

CAPABILITY	CLASSIFICATION
Query execution (DuckDB)	Custom-built on an embedded engine; maps to native warehouse SQL
Semantic contract	Custom-built; would be *configured* on dbt/Unity Catalog
Row/column/domain security	Custom-built; would be *native* on Snowflake/Unity Catalog
Text retrieval (BM25)	Custom-built; would be *native* via Cortex Search
Causal inference, ladder, ledger	**Custom-built — no native equivalent exists**
Playbook memory	**Custom-built — no native equivalent exists**
Narration	**Externally integrated** (Anthropic API), optional
UI	Custom-built

The two rows marked "no native equivalent" are the actual product.

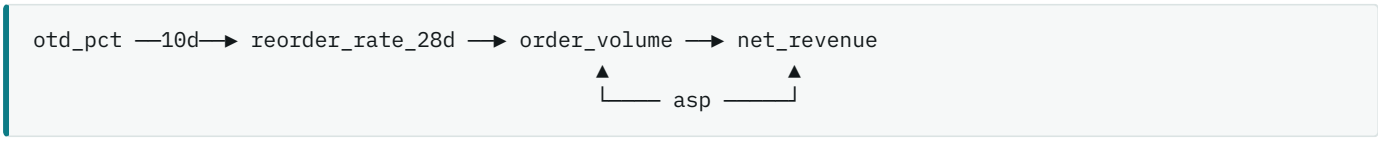
Part D — Minimum Prototype Expectations

These are the hard requirements. **All ten are delivered.**

• Three to five connected KPIs across two or three data sources with different grains or refresh cadences

DELIVERED — exceeds

Five KPIs across four sources, three grains, four cadences. They are causally connected, not merely co-located:



Honest caveat: a single `analyse` run targets **one** KPI. The connection is exercised by the mechanism ledger (which measures four KPIs in one run) and by the sweep (which scans all five), but there is no simultaneous multi-KPI diagnosis.

• A lightweight KPI or semantic contract covering definitions, calculations, drivers, thresholds, lineage and access restrictions

DELIVERED — all six elements

`config/kpi_contract.yaml`, 190 lines: definitions with label/unit/grain, calculations as explicit SQL, drivers per KPI, materiality thresholds and sigma bands, lineage strings, and access class per KPI. Enforced — `get_kpi()` raises on anything undeclared, which is what stops a model inventing a metric.

• At least two personas receiving different insight narratives or recommended actions

DELIVERED — four personas

CFO, RSM North, Supply Chain Lead, Analyst. Different narratives *and* different recommendations *and* different evidence access. A test asserts three are byte-different. Caveat on template-driven differentiation noted in objective 4.

• One multi-factor KPI movement with known or simulated underlying drivers

DELIVERED

S1. Planted: a WH-3 dispatch SLA collapse propagating to three named accounts at a 10-day lag, **plus** a Discount-tier mix shift. Recovered: service failure at 80% explanatory power (L3, DiD −20.6%, p<0.0001, clean placebo) and the mix effect at 10%, with the identity closing to under ₹1. The engine **independently rediscovered the 10-day lag** it was given in the graph.

• One low-confidence scenario in which the engine requests clarification or abstains

DELIVERED

S2. A competitor promotion and our own price rise start the same day in the same region, so the cross-channel control is contaminated by design. Both hypotheses stall at L1, 83% of retrieved evidence supports a rival theme, and the engine returns `COMPETING` with a priced separating test (₹1.8L, 14 days, owner CFO) instead of a guess.

• One sparse-history or newly launched KPI scenario

DELIVERED

S3. 19 days of history, peer-group prior, widened interval, `INSUFFICIENT_HISTORY`, no causal claim.

• One role-based security or entitlement scenario

DELIVERED — three distinct demonstrations

RSM North querying West is refused at pre-flight before any SQL runs. Supply Chain receives a rupee-free narrative with packet-level redaction. CFO is denied CRM verbatims and reaches L3 through the counterfactual instead.

• Evidence showing source freshness, analytical method, contribution, confidence and lineage

DELIVERED — all five on every response

Freshness per source with SLA and breach state; method declared per step with a reason; contribution as explanatory power and share shift; confidence as ladder grade plus a basis string; lineage per KPI.

• A clear breakdown of LLM versus non-LLM processing

DELIVERED

A method registry where every step declares its type and *why that type was chosen*. Rendered in the UI and in `telemetry.method_mix`. 100% non-LLM offline, 91% with narration. **Caveat:** one declared category — ML — is never used.

• Runtime telemetry covering latency, model calls, token usage and estimated cost

DELIVERED — all four

```
fitness 18.5ms → sift 4.9ms → split 7.9ms → source 388.6ms → propagate 31.5ms
                                          → solve 0.1ms → narrate 0.0ms
total 453 ms | 1 model call | 1,066 in / 184 out tokens | ₹0.17 | 91% non-LLM
```

Caveat: the `cached` field in this record is always False — see LLM economics.

Part E — The honest summary

Score

GROUP	DELIVERED	PARTIAL	NOT BUILT
8 objectives	5	3	0
LLM-not-truth statement	1	0	0
10 complexities	4	6	0
8 solutioning areas	2	6	0
10 minimum expectations	**10**	**0**	**0**

The seven things that are genuinely absent

1. **Forecasting** — no forward projection of any kind.
2. **Traditional ML** — zero. An empty category in my own method ledger.
3. **Caching** — a field that is always False.
4. **Drift detection** — none, for data or model.
5. **Confidence calibration** — scored, never validated.
6. **Delivery** — channels declared, nothing sent.
7. **Scalability evidence** — 45k rows, no load test.

The four dead-config items — fix these first

These are worse than gaps because they read as features to anyone browsing the repo:

1. `fiscal_calendar: "apr_mar"` — declared, never read by any code.
2. `hierarchy: [region, city]` — declared, and `city` **does not exist in the data**.
3. `correction` — accepted by the API, stored, never read back.
4. `cached: bool` — present in every telemetry record, always False.

What I would defend without hesitation

The evidence ladder and its refusals. The numeric guard. Entitlement enforcement before prompt assembly. The mechanism ledger with lag alignment. Conflict-scored corroboration. Suppression accounting. The backtest reporting its own false-alarm rate. All ten minimum expectations. And the seven defects already found and fixed, which are documented rather than buried.

Where improvement should branch, in priority order

1. **Kill the four dead-config items** — either implement or delete. Half a day.
2. **Persist runs** for real auditability. Currently the weakest part of a claim I lean on.
3. **Correction workflow** — the highest-value missing loop; the field already exists.
4. **A conflicting second KPI definition** to actually demonstrate reconciliation.

5. **Confidence calibration harness** — predicted vs realised, a reliability curve.
6. **Scale evidence** — a 10M-row estate and a published latency curve.
7. **Forecasting / "if we do nothing" trajectory** — the most visible missing capability.
8. **Delivery** — even one working channel would make the personalization claim true.